OBJECT™
TECHNOLOGIES

Java IO

What will be covered

OBJECT TECHNOLOGIES

- Buffered stream
- Flushing buffered straems
- Serialization
- Object streams
- Use of serialization
- Example
- Desrialization
- Externalizable

Buffered streams

OBJECT

- Each read or write request is handled directly by the underlying OS.
- This can make a program much less efficient, This can make a program much less efficient, since each such request often triggers disk access, network activity, or some other operation that is relatively expensive.
- To reduce this kind of overhead, the Java platform implements *buffered* I/O streams.
- Buffered input streams read data from a memory area known as a *buffer*; the native input API is called only when the buffer is empty.
- Similarly, buffered output streams write data to a buffer, and the native output API is called only when the buffer is full
- Buffered stream classes :
  - BufferedInputStream
  - BufferedOutputstream
  - BufferedReader
  - BufferedWriter

Buffered Stream - example

```
//create file object
File file = new File("C://FileIO//ReadFile.txt");
BufferedInputStream bin = null;

try
{
    //create FileInputStream object
    FileInputStream fin = new FileInputStream(file);

    //create object of BufferedInputStream
    bin = new BufferedInputStream(fin);

    //create a byte array
    byte[] contents = new byte[1024];

    int bytesRead=0;
    String strFileContents;

    while( (bytesRead = bin.read(contents)) != -1){

        strFileContents = new String(contents, 0, bytesRead);
        System.out.print(strFileContents);
    }
```

Actual read operation is performed by buffered stream. For creating buffered stream object instance of File stream is passed as an argument

Buffered Stream - example

OBJECT TECHNOLOGIES

```
public static void main(String[] args) throws Exception {
    FileWriter writer = new FileWriter("D:\\testout.txt");
    BufferedWriter buffer = new BufferedWriter(writer);
    buffer.write("Welcome to javaTpoint.");
    buffer.close();
    System.out.println("Success");
    }
}
```

- Writing operation is performed by buffered stream
- It will write the data into the buffer and when buffer is full then only it is written in the underlying stream i.e. file.
- Size of the buffer can be configured at the time of creating buffered streams for performance optimization.

Flushing Buffered Streams

OBJECT TECHNOLOGIES

- It often makes sense to write out a buffer at critical points, without waiting for it to fill. This is known as *flushing* the buffer.
- Some buffered output classes support *autoflush*, specified by an optional constructor argument. When autoflush is enabled, certain key events cause the buffer to be flushed. For example, an autoflush PrintWriter object flushes the buffer on every invocation of println or format.
- To flush a stream manually, invoke its flush method.
- The flush method is valid on any output stream, but has no effect unless the stream is buffered.

Serialization

OBJECT TECHNOLOGIES

- To *serialize* an object means to convert its state to a byte stream so that the byte stream can be reverted back into a copy of the object.
- A Java object is *serializable* if its class or any of its superclasses implements either the java.io.Serializable interface or its subinterface, java.io.Externalizable.
- *Deserialization* is the process of converting the serialized form of an object back into a copy of the object.
- All the instance variables are saved during serialization
- Generally derived values are not serialized, if any instance member is to be excluded from serialization, it can be marked as transient.
- If Super class implements Serializable then sub class are also Serializable automatically.

Object streams

OBJECT TECHNOLOGIES

- object streams support I/O of objects.
- Most, but not all, standard classes support serialization of their objects.
- Those that do implement the marker interface Serializable.
- The object stream classes are ObjectInputStream and ObjectOutputStream. These classes implement ObjectInput and ObjectOutput, which are subinterfaces of DataInput and DataOutput.
- That means that all the primitive data I/O methods are also implemented in object streams. So an object stream can contain a mixture of primitive and object values.
- The **Java.io.ObjectOutputStream** class writes primitive data types and graphs of Java objects to an OutputStream.The objects can be read (reconstituted) using an ObjectInputStream

Use of Serialization

OBJECT TECHNOLOGIES

- serialization will translate the Object state to Byte Streams. This Byte stream can be used for different purpose.
  - Write to Disk
  - Store in Memory
  - Sent byte stream to other platform over network
  - Save byte stream in DB(As BLOB)
- It is mainly used in Hibernate, RMI, JPA, EJB and JMS technologies.
- It is mainly used to travel object's state on the network (known as marshaling).

Serialization example

OBJECT TECHNOLOGIES

```
import java.io.*;
class Persist{
 public static void main(String args[])throws Exception{
  Student s1 =new Student(211,"ravi");

  FileOutputStream fout=new FileOutputStream("f.txt");
  ObjectOutputStream out=new ObjectOutputStream(fout);

  out.writeObject(s1);
  out.flush();
  System.out.println("success");
 }
}
```

- Note : If Student class does not implement Serializable interface, it will throw NotSerializableException at run time

Deserialization example

OBJECT

```
import java.io.*;
class Depersist{
 public static void main(String args[])throws Exception{

  ObjectInputStream in=new ObjectInputStream(new FileInputStream("f.txt"));
  Student s=(Student)in.readObject();
  System.out.println(s.id+" "+s.name);

  in.close();
 }
}
```

- Note : readObject() method returns Object that has to be upcasted to an appropriate type. If needed it can be supported with

Externalizable in java

OBJECT TECHNOLOGIES

- The Externalizable interface provides the facility of writing the state of an object into a byte stream in compress format. It is not a marker interface.
- The Externalizable interface provides two methods:
  - **public void writeExternal(ObjectOutput out) throws IOException**
  - **public void readExternal(ObjectInput in) throws IOException**

Streams and File IO-II

In this chapter we will learn about using buffering while performing reading and writing using streams which will affect the overall performance. We will even learn about the process of serialization and object streams that help in serialization.

Buffered Streams

Using stream classes directly, without any buffering, results in very inefficient reading or writing of streams in turn making you program much less efficient, since each such request often triggers disk access, network activity, or some other operation that is relatively expensive. To make IO operations more efficient the Java platform implements buffered I/O streams.

Buffered streams in Java

Buffered input streams read data from a memory area known as a buffer; the native input API is called only when the buffer is empty.

Similarly, buffered output streams write data to a buffer, and the native output API is called only when the buffer is full.

Buffered stream wraps the unbuffered stream to convert it into a buffered stream. For that unbuffered stream object is passed to the constructor for a buffered stream class. For example wrapping FileReader and FileWriter to get BufferedReader and BufferedWriter.

Buffered streams classes in Java

There are four buffered stream classes-

BufferedInputStream and BufferedOutputStream are used to wrap unbuffered byte streams to create buffered byte streams.

BufferedReader and BufferedWriter are used to wrap unbuffered character streams to create buffered character streams.

Flushing buffered streams

Since the characters or bytes are stored in a buffer before writing to the file so flushing the buffer is required sometimes without waiting for it to fill. There is an explicit flush() method for doing that. In most buffered stream implementations even close() method internally calls flush() method before closing the stream. Note that the flush method is valid on any output stream, but has no effect unless the stream is buffered.

Writing in file using buffered stream :

```
BufferedReader br = null;
BufferedWriter bw = null;
```

Writing in file using buffered stream :

```
BufferedReader br = null;
BufferedWriter bw = null;
try
{
   bw = new BufferedWriter(new FileWriter("E:/fileiodemos/buffer.txt"));
   br = new BufferedReader(new InputStreamReader(System.in));
   System.out.println("Enter 'stop' to quit");
   String str;
   while(!(str = br.readLine()).equals("stop"))
   {
      bw.write(str);
      bw.newLine();
   }
}
catch(IOException e)
{
   e.printStackTrace();
}
finally
{
   try {
      bw.close();
      br.close();
   } catch (IOException e) {
      e.printStackTrace();
   }
}
```

Reading from file using buffered stream :

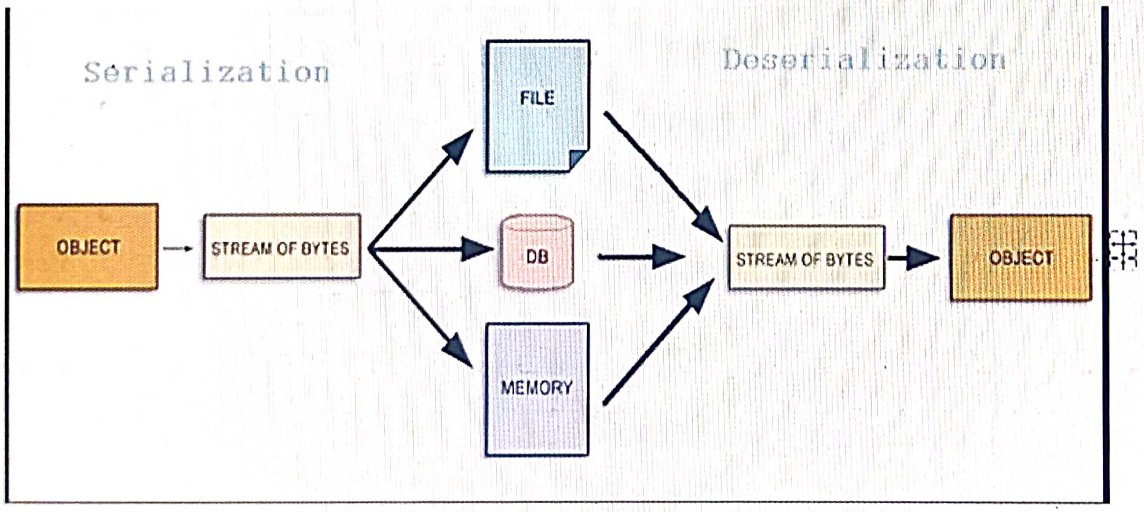
```
BufferedReader br = new BufferedReader(new FileReader("E:/fileiodemos/buffer.txt"));
String str;
int cnt = 0;
while((str=br.readLine())!=null)
{
   cnt++;
```

Reading from file using buffered stream :

```
BufferedReader br = new BufferedReader(new FileReader("E:/fileiodemos/buffer.txt"));
String str;
int cnt = 0;
while((str=br.readLine())!=null)
{
  cnt++;
  System.out.println(cnt+"."+str+" - "+str.length()+" chars");
}
br.close();
```

Serialization

Serialization is the process of converting an object into a stream of bytes to store the object or transmit it to memory, a database, or a file. Its main purpose is to save the state of an object in order to be able to recreate it when needed. The reverse process is called deserialization.



How serialization works? Which kinds of object are eligible for serialization?

1. Only objects of classes that implement the java.io.Serializable interface can be written to and read from an output/input stream.
2. Serializable is a marker interface, which doesn?t define any methods. Only objects that are marked 'serializable' can be used with ObjectOutputStream and ObjectInputStream.

How serialization works? Which kinds of object are eligible for serialization?

1. Only objects of classes that implement the java.io.Serializable interface can be written to and read from an output/input stream.
2. Serializable is a marker interface, which doesn?t define any methods. Only objects that are marked 'serializable' can be used with ObjectOutputStream and ObjectInputStream.
3. Most classes in Java (including Date and primitive wrappers Integer, Double, Long, etc) implement the Serializable interface.
4. If you attempt to write an object of a non-serializable class, you will get a java.io.NotSerializableException.

We need Serialization for the following reasons:

- **Communication:** Serialization involves the procedure of object serialization and transmission. This enables multiple computer systems to design, share and execute objects simultaneously.
- **Caching:** The time consumed in building an object is more compared to the time required for de-serializing it. Serialization minimizes time consumption by caching the giant objects.
- **Deep Copy:** Cloning process is made simple by using Serialization. An exact replica of an object is obtained by serializing the object to a byte array, and then de-serializing it.
- **Cross JVM Synchronization:** The major advantage of Serialization is that it works across different JVMs that might be running on different architectures or Operating Systems.
- **Persistence:** The State of any object can be directly stored by applying Serialization on to it and stored in a database so that it can be retrieved later.

Points to remember :

1. It must implements the Serializable interface. Otherwise, we will get java.io.Not SerializableException when trying to serialize an object of the class.
2. A constant named serialVersionUID is declared and assigned a long value:private static final long serialVersionUID = 1234L; This is a conventional constant which should be declared when a class implements the Serializable interface. The serial version UID strongly ensures compatibility between the serialized and de-serialized versions of objects of a class, because the process of serialization and de-serialization can happen on different computers and systems. Although this declaration is optional, it's always recommended to declare the serialVersionUID for a serializable class.
3. When a variable is marked as transient, its object won't be serialized during serialization.

Object Streams

Object Streams

Just as data streams support I/O of primitive data types, object streams support I/O of objects. Most, but not all, standard classes support serialization of their objects. Those that do implement the marker interface Serializable.

- Objects are serialized by object output streams.
- They are deserialized by object input streams.
- These are instances of java.io.ObjectOutputStream and java.io.ObjectInputStream, respectively.
- The object stream classes are ObjectInputStream and ObjectOutputStream.
- These classes implement ObjectInput and ObjectOutput, which are subinterfaces of DataInput and DataOutput. That means that all the primitive data I/O methods covered in Data Streams are also implemented in object streams.

So an object stream can contain a mixture of primitive and object values. To write an object onto a stream, we have to chain an ObjectOutputStream with instance of OutputStream, then pass the object to the ObjectOutputStream's writeObject() method. Similarly readObject() method is used on ObjectInputStream object which returns Object. To read an object from a stream, we have to chain an ObjectInputStream with instance of InputStream,

Example for Serialization :

```
Emp [] allemp = new Emp[3];
allemp[0] = new SalesManager("Priya",11,11,1999,1001,10000.00,100000.00,4.3,10);
allemp[1] = new Programmer("Amol",12,12,1998,1002,12000.00,45.5,870.00,8);
allemp[2] = new Admin("Vikas",13,9,1997,1003,13000.00,980.00);
//serialization
ObjectOutputStream oos = null;
try
{
   oos = new ObjectOutputStream(new FileOutputStream("E:/fileiodemos/emps.dat"));
   for(Emp e : allemp)
      oos.writeObject(e);
   System.out.println("Emps are written");
}
catch(Exception e)
{
   e.printStackTrace();
}
finally
```

Example for Serialization :

```
Emp [] allemp = new Emp[3];
allemp[0] = new SalesManager("Priya",11,11,1999,1001,10000.00,100000.00,4.3,10);
allemp[1] = new Programmer("Amol",12,12,1998,1002,12000.00,45.5,870.00,8);
allemp[2] = new Admin("Vikas",13,9,1997,1003,13000.00,980.00);
//serialization
ObjectOutputStream oos = null;
try
{
   oos = new ObjectOutputStream(new FileOutputStream("E:/fileiodemos/emps.dat"));
   for(Emp e : allemp)
      oos.writeObject(e);
   System.out.println("Emps are written");
}
catch(Exception e)
{
   e.printStackTrace();
}
finally
{
   try {
      oos.close();
   } catch (IOException e) {
       e.printStackTrace();
   }
}
```

Example for Deserialization :

```
ObjectInputStream ois = new ObjectInputStream(new FileInputStream("E:/fileiodemos/emps.dat"));
//when we exactly know number of objects to be read
/*Emp [] empty = new Emp[3];
for(int i=0;i<empty.length;i++)
{
   empty[i] = (Emp)ois.readObject();
}
```

Example for Deserialization :

```
ObjectInputStream ois = new ObjectInputStream(new FileInputStream("E:/fileiodemos/emps.dat"));
//when we exactly know number of objects to be read
/*Emp [] empty = new Emp[3];
for(int i=0;i<empty.length;i++)
{
    empty[i] = (Emp)ois.readObject();
}
for(Emp e : empty)
{
   System.out.println(e);
   System.out.println("**********");
}*/
//when we exactly don't know number of objects to be read
List<Emp> emps = new ArrayList<>();
while(true)
{
   try
   {
      Emp e = (Emp)ois.readObject();
      emps.add(e);
   }
   catch(Exception e)
   {
      break;
   }
}
for(Emp e : emps)
{
   System.out.println(e);
   System.out.println("**********");
}
```

Assignments

1. Accept the file name from the user. Check whether file is a directory or file. If it is a directory, list all the files in it and if it is a

Assignments

1. Accept the file name from the user. Check whether file is a directory or file. If it is a directory, list all the files in it and if it is a file, check the size. If the size of the file is more than 25, use buffering to read the contents otherwise use file stream to read the content.
2. Display line no with max no of characters within a file. Accept file name from the user.
3. Write separate programs for following
   a) Create an array of 3 employees. Save all the employess in a file.
   b) Read all the employees back from file and display on the console.
4. Display the contents of file right aligned to the line with max length. Display '-' for all spaces while aligning to the right side. E.g.

   ----------hello
   ----------uses file streams
   ---------This is not line with max chars
   object oriented programming in java
   ----------welcome to know-it
5. Create a class Student extending from Person and add data members course(Course) and PRN(String) and age(int). Course is another class which has data members as courseid(int), coursename(String) and fees(double). Note student has a course (has a relationship). Declare age data member as transient. Create array of 3 students and serialize in the file.